5 - Excepciones

Tabla de contenidos

1 Introducción 2
2 Diferentes tipos de “errores” 3
  2.1 Errores de sintaxis 3
  2.2 Excepciones durante la ejecución 3
  2.3 Errores lógicos o *bugs* 4
  2.4 *Warnings* 4
3 Cómo leer un *traceback* 4
4 Tipos de errores *built-in* 5
5 Lanzamiento de excepciones 7
  5.1 Cómo construir una excepción 7
  5.2 Qué ocurre al lanzar una excepción 8
6 Manejo de excepciones 9
  6.1 Uso del objeto excepción 9
  6.2 Múltiples bloques `except` 10
  6.3 Múltiples tipos en un mismo `except` 10
  6.4 Bloque `else` 10
  6.5 Volver a intentar una operación 11
  6.6 Bloque `finally` 11
  6.7 Ejemplo final 12
7 Excepciones a medida 13
  7.1 Las excepciones son objetos 13
  7.2 Jerarquía de excepciones 14
  7.3 Orden de excepciones por especificidad 15
8 *Warnings* 15
  8.1 Crear una advertencia 15
  8.2 Categorías frecuentes 16
  8.3 Advertencias personalizadas 16
  8.4 Filtrar advertencias 16
9 Tópicos avanzados 🧩 17
  9.1 Agregar información a la excepción 17
  9.2 Propagación avanzada 18
    9.2.a Relanzar una excepción 18
    9.2.b Transformar y encadenar excepciones 19
  9.3 Administrar *warnings* temporalmente 19

1 Introducción

Al programar es inevitable encontrarse con situaciones que impiden que una operación pueda completarse. A veces el problema está en el código: olvidamos cerrar un paréntesis, escribimos mal el nombre de una variable o intentamos usar un método que un objeto no posee. Otras veces el código es correcto, pero recibe un dato con el que no puede trabajar: un divisor igual a cero, una posición que no existe en una lista o la ruta a un archivo inexistente.

Hasta ahora, cuando una función detectaba una situación inválida, muchas veces la indicábamos con un `print`:

```
class CuentaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def extraer(self, monto):
        if monto > self.saldo:
            print("No hay saldo suficiente")
            return

        self.saldo -= monto
        return monto
```

Este código informa el problema a una persona que esté mirando la pantalla, pero no se lo comunica de una manera útil al resto del programa. Además, la clase `CuentaBancaria` queda obligada a elegir cómo mostrar el mensaje, aunque quizás se la quiera usar en un sitio web, una aplicación gráfica o un programa que no tenga consola.

Las **excepciones**, en particular los errores, permiten comunicar estas situaciones de manera explícita:

```
class CuentaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def extraer(self, monto):
        if monto > self.saldo:
            raise ValueError("No hay saldo suficiente")

        self.saldo -= monto
        return monto
```

Ahora, el método detecta el problema y **lanza una excepción**. La parte del programa que utiliza la cuenta puede decidir si muestra un mensaje, solicita otro monto, registra el incidente o deja que la ejecución termine.

> **!** Más que un color alarmante
>
> Una excepción no es solamente un texto impreso en color rojo. Es un objeto que representa una situación excepcional y que modifica el flujo normal de ejecución del programa.

2 Diferentes tipos de "errores"

En una conversación informal solemos llamar **error** a cualquier cosa que salió mal. Sin embargo, al programar conviene distinguir varias situaciones.

2.1 Errores de sintaxis

Un **error de sintaxis** ocurre cuando el código no respeta la gramática de Python y, por lo tanto, el intérprete no puede comenzar a ejecutarlo.

```
print("Hola"
```

```
  File "programa.py", line 1
    print("Hola"
          ^
SyntaxError: '(' was never closed
```

Python informa el archivo, la línea aproximada y la razón del problema. Cuando el error se encuentra en el archivo que se intenta ejecutar, normalmente no tiene sentido manejarlo desde ese mismo archivo: primero es necesario corregir el código para que pueda ejecutarse.

2.2 Excepciones durante la ejecución

Una **excepción** ocurre mientras Python está ejecutando el programa. El código puede ser sintácticamente válido y, aun así, encontrar una operación que no puede completar.

```
dividendo = 10
divisor = 0
resultado = dividendo / divisor
```

```
Traceback (most recent call last):
  File "programa.py", line 3, in <module>
    resultado = dividendo / divisor
                ~~~~~~~~~~^~~~~~~~~
ZeroDivisionError: division by zero
```

En este ejemplo, Python crea un objeto de tipo `ZeroDivisionError` y lo lanza o eleva. **Capturar una excepción** significa interceptarla mientras se propaga para ejecutar una respuesta alternativa, en lugar de dejar que termine el programa. Si ninguna parte del programa la captura, la ejecución termina y se muestra el *traceback*. Más adelante veremos cómo hacerlo con `try` y `except`.

2.3 Errores lógicos o *bugs*

Un **error lógico**, también llamado *bug*, no necesariamente produce una excepción. El programa puede ejecutarse hasta el final y devolver un resultado equivocado.

```
def promedio(numeros):
    return sum(numeros) / (len(numeros) + 1)
```

Si se llama a la función con una lista no vacía de números, la misma se ejecuta sin lanzar una excepción, pero el cálculo es incorrecto. En este caso es necesario revisar la implementación del algoritmo.

2.4 *Warnings*

Un ***warning*** o advertencia comunica una situación que merece atención, pero normalmente no interrumpe el programa. Por ejemplo, un paquete puede advertir que una función o método continuará funcionando por el momento, aunque será eliminado en una versión futura.

Más adelante veremos cómo crear y filtrar estas advertencias.

3 Cómo leer un *traceback*

Cuando una excepción no es capturada, Python muestra un ***traceback***. Aunque al principio puede parecer intimidante, contiene información muy útil para investigar lo ocurrido.

Consideremos el siguiente programa:

```
def convertir_a_entero(texto):
    return int(texto)

def duplicar(texto):
    numero = convertir_a_entero(texto)
    return numero * 2

print(duplicar("diez"))
```

Al ejecutarlo obtenemos un mensaje similar al siguiente:

```
Traceback (most recent call last):
  File "programa.py", line 8, in <module>
    print(duplicar("diez"))
          ^^^^^^^^^^^^^^^^
  File "programa.py", line 5, in duplicar
    numero = convertir_a_entero(texto)
             ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "programa.py", line 2, in convertir_a_entero
    return int(texto)
           ^^^^^^^^^^
ValueError: invalid literal for int() with base 10: 'diez'
```

El *traceback* contiene:

1. La secuencia de llamadas: indica el camino que siguió el programa. En este caso, el código principal llamó a `duplicar`, que a su vez llamó a `convertir_a_entero`.
2. El archivo y el número de línea: ayudan a ubicar cada llamada involucrada.
3. La instrucción ejecutada: muestra el código relacionado con el problema.
4. El tipo de excepción: aquí es `ValueError`.
5. El mensaje: explica qué valor no pudo convertirse: `'diez'`.

Una estrategia útil consiste en comenzar por la última línea, para conocer el tipo y el mensaje, y luego recorrer de abajo hacia arriba las llamadas que llevaron hasta el problema. La última línea de código mencionada suele indicar dónde se lanzó la excepción, pero la causa conceptual puede estar antes. Por ejemplo, el dato inválido podría haberse originado en otra función.

4 Tipos de errores *built-in*

Python incluye una jerarquía amplia de excepciones. No es necesario memorizarla de manera completa, pero sí conviene reconocer algunos tipos habituales y prestar atención al mensaje que los acompaña.

Excepción	Situación frecuente
`SyntaxError`	El código no respeta la sintaxis de Python.
`NameError`	Se intenta usar un nombre que no está definido.
`TypeError`	El tipo del objeto no admite la operación solicitada.
`ValueError`	El tipo es apropiado, pero el valor no es válido para la operación.
`IndexError`	Se intenta acceder a una posición inexistente de una secuencia.
`KeyError`	Se solicita una clave inexistente de un diccionario.
`AttributeError`	El objeto no posee el atributo o método solicitado.
`ZeroDivisionError`	Se intenta dividir por cero.
`FileNotFoundError`	No se encuentra un archivo o directorio solicitado.
`ModuleNotFoundError`	Python no puede encontrar un módulo importado.

Veamos algunos ejemplos:

```
print(nombre)
```

```
NameError: name 'nombre' is not defined
```

```
5 + "5"
```

```
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

```
int("cinco")
```

```
ValueError: invalid literal for int() with base 10: 'cinco'
```

```
numeros = [10, 20, 30]
numeros[8]
```

```
IndexError: list index out of range
```

```
persona = {"nombre": "Ana"}
persona["edad"]
```

```
KeyError: 'edad'
```

```
numero = 42
numero.append(5)
```

```
AttributeError: 'int' object has no attribute 'append'
```

i ¿TypeError o ValueError? 🤔

Usamos `TypeError` cuando el objeto es de un tipo que no admite la operación. Por ejemplo, una edad expresada como `"veinte"` cuando se espera un número entero. En cambio, usamos `ValueError` cuando el tipo es correcto, pero el valor no es válido. Por ejemplo, una edad igual a `-20`.

Por ejemplo:

```
def validar_edad(edad):
    if not isinstance(edad, int):
        raise TypeError("La edad debe ser un número entero")
    if edad < 0:
        raise ValueError("La edad no puede ser negativa")
```

5 Lanzamiento de excepciones

Python lanza automáticamente excepciones cuando no puede completar ciertas operaciones. También podemos lanzarlas nosotros deliberadamente con la instrucción `raise` cuando nuestro programa detecta una condición inválida.

La forma más habitual es:

```
raise TipoDeExcepcion("Mensaje que explica el problema")
```

Por ejemplo:

```
def calcular_descuento(precio, porcentaje):
    if precio < 0:
        raise ValueError("El precio no puede ser negativo")

    if not 0 <= porcentaje <= 100:
        raise ValueError("El porcentaje debe estar entre 0 y 100")

    return precio * porcentaje / 100
```

Si los argumentos son válidos, la función devuelve el descuento normalmente:

```
calcular_descuento(2000, 10)
```

```
200.0
```

Si no lo son, la ejecución de la función se interrumpe en el `raise`:

```
calcular_descuento(2000, 120)
```

```
ValueError: El porcentaje debe estar entre 0 y 100
```

La función no necesita imprimir nada ni decidir cómo se presentará el problema. Su responsabilidad es señalar que no puede cumplir su contrato con los datos recibidos.

5.1 Cómo construir una excepción

Una excepción debería aportar dos piezas de información:

- **el tipo** permite que el programa distinga clases de problemas;
- **el mensaje** ofrece detalles para que una persona pueda comprender el caso concreto.

`Exception` es la clase base de la que heredan casi todas las excepciones que representan fallos ordinarios de un programa, como `ValueError`, `TypeError` o `FileNotFoundError`. Por eso es posible crear y lanzar directamente una excepción general:

```
raise Exception("El porcentaje es incorrecto")
```

Sin embargo, su tipo sólo comunica que ocurrió algún problema. Siempre que exista una alternativa adecuada, es preferible lanzar una excepción más específica:

```
raise ValueError("El porcentaje debe estar entre 0 y 100")
```

El mensaje debe explicar qué condición no se cumplió. Cuando resulte útil, puede incluir el valor problemático:

```
raise ValueError(f"El porcentaje debe estar entre 0 y 100; se recibió
{porcentaje}")
```

5.2 Qué ocurre al lanzar una excepción

Cuando se ejecuta `raise`, Python interrumpe inmediatamente el flujo normal del programa y no se ejecutan las instrucciones posteriores de la función actual. En su lugar, Python busca un bloque `except` que pueda capturar la excepción. Mientras lo busca, también abandona las funciones que habían realizado las llamadas pendientes. Si no encuentra un `except` apropiado, el programa termina y muestra el *traceback*.

```
def raiz_cuadrada(numero):
    if numero < 0:
        raise ValueError("No se calcula la raíz real de un número negativo")
        print("Esta línea nunca se ejecuta")

    return numero ** 0.5
```

Para un número negativo no se ejecuta ni el `print` posterior al `raise` ni el `return` de la función. La única forma de continuar la ejecución de manera controlada es que alguna parte del programa capture la excepción.

Este recorrido de la excepción hacia las funciones que realizaron las llamadas se denomina **propagación**. Podemos observarlo con varias funciones encadenadas:

```
def convertir(texto):
    return int(texto)

def leer_cantidad(texto):
    return convertir(texto)

def preparar_pedido(texto):
    cantidad = leer_cantidad(texto)
    return f"Se prepararán {cantidad} unidades"
```

```
preparar_pedido("muchas")
```

`int` lanza un `ValueError` y la ejecución de `convertir` se interrumpe. Como `convertir` no captura la excepción, esta se propaga a `leer_cantidad`, cuya ejecución también se interrumpe. Lo mismo ocurre con `preparar_pedido`: la excepción atraviesa todas las llamadas y llega al código principal. Como allí tampoco existe un bloque capaz de manejarla, el programa termina y muestra el *traceback*.

6 Manejo de excepciones

Para **capturar** una excepción, se encierra la operación que podría lanzarla en un bloque `try` y se indica en un bloque `except` qué tipo de excepción se espera y cómo responder. De esta manera, la excepción deja de propagarse y el programa puede continuar de forma controlada.

```
try:
    numero = int(input("Ingrese un número: "))
except ValueError:
    print("La entrada no representa un número entero")
```

Python ejecuta primero el bloque `try`:

- si no se lanza una excepción, omite el bloque `except` y continúa;
- si se lanza un `ValueError`, interrumpe el `try`, ejecuta el `except` correspondiente y después continúa con el código que se encuentre luego de la estructura;
- si se lanza una excepción de otro tipo, este `except` no la captura y la excepción continúa propagándose.

> **i** Nota
>
> Decimos que `except` **captura** la excepción y que su bloque **maneja** la situación. Capturarla no implica necesariamente que el problema haya desaparecido: el bloque debe tomar una decisión coherente, como solicitar otro dato, usar una alternativa o terminar una operación de manera controlada.

6.1 Uso del objeto excepción

La excepción lanzada es un objeto. Podemos asignarle un nombre mediante `as` y consultar su mensaje:

```
try:
    numero = int("cuarenta")
except ValueError as error:
    print("No se pudo realizar la conversión")
    print("Detalle:", error)
```

```
No se pudo realizar la conversión
Detalle: invalid literal for int() with base 10: 'cuarenta'
```

Durante la ejecución del bloque `except`, el nombre `error` referencia al objeto capturado. Al finalizar el bloque, Python elimina esa asociación. También se usan con frecuencia los nombres `excepcion` o `e`.

6.2 Múltiples bloques `except`

Una operación puede fallar de distintas maneras. Los bloques separados permiten responder de forma específica a cada tipo:

```
try:
    posicion = int(input("Ingrese una posición: "))
    colores = ["rojo", "verde", "azul"]
    print(colores[posicion])
except ValueError:
    print("La posición debe ser un número entero")
except IndexError:
    print("La posición no existe en la lista")
```

Python prueba los bloques `except` de arriba hacia abajo y ejecuta solamente el primero cuyo tipo coincida con la excepción.

6.3 Múltiples tipos en un mismo except

Si distintas excepciones requieren exactamente la misma respuesta, pueden agruparse en una tupla:

```
try:
    numero = int(input("Ingrese un divisor: "))
    resultado = 100 / numero
except (ValueError, ZeroDivisionError) as error:
    print("No se pudo realizar el cálculo:", error)
```

Agrupar excepciones reduce la repetición, pero también pierde la posibilidad de dar una respuesta diferente a cada problema. Conviene hacerlo únicamente cuando el manejo sea realmente el mismo.

6.4 Bloque `else`

El bloque `else` se ejecuta solamente si el `try` finaliza sin lanzar ninguna excepción:

```
try:
    numero = int(input("Ingrese un divisor: "))
    resultado = 100 / numero
except ValueError:
    print("La entrada debe ser un número entero")
```

```
except ZeroDivisionError:
    print("El divisor no puede ser cero")
else:
    print("Resultado:", resultado)
```

Podríamos colocar el último `print` dentro del `try`, pero usar `else` permite que el bloque `try` contenga sólo las instrucciones que esperamos que produzcan las excepciones capturadas.

6.5 Volver a intentar una operación

En programas interactivos, muchas veces una entrada inválida no requiere terminar la ejecución: podemos informar el problema y solicitar nuevamente el dato. Un bucle `while` permite repetir el intento hasta obtener un valor válido:

```
while True:
    try:
        numero = int(input("Ingrese un número entero: "))
    except ValueError:
        print("La entrada no es válida. Intente nuevamente.")
    else:
        break

print("Número ingresado:", numero)
```

Si la conversión lanza un `ValueError`, se muestra el mensaje y comienza una nueva iteración. El `break` sólo se ejecuta cuando el bloque `try` finaliza correctamente, por lo que el bucle termina una vez que se ha ingresado un número entero.

6.6 Bloque `finally`

El bloque `finally` se ejecuta siempre, tanto si ocurrió una excepción como si no ocurrió:

```
archivo = open("datos.txt", encoding="utf-8")

try:
    contenido = archivo.read()
    numero = int(contenido)
except ValueError:
    print("El archivo no contiene un número entero")
finally:
    archivo.close()
```

Su uso principal es liberar recursos o realizar tareas de limpieza que deben suceder bajo cualquier resultado. En el caso particular de los archivos, normalmente es preferible usar un *context manager* que se ocupa del cierre automáticamente:

```python
with open("datos.txt", encoding="utf-8") as archivo:
    contenido = archivo.read()
```

`finally` sigue siendo útil cuando la adquisición y liberación del recurso no puede expresarse con un *context manager*.

6.7 Ejemplo final

Una estructura puede contener `except`, `else` y `finally`:

```python
try:
    texto = input("Ingrese un número: ")
    numero = int(texto)
except ValueError as error:
    print("Entrada inválida:", error)
else:
    print("El doble es", numero * 2)
finally:
    print("Fin del intento")
```

La estructura comienza siempre con `try`. A continuación puede incluir uno o más bloques `except` y, si estos existen, un bloque `else` opcional. Al final puede agregarse un bloque `finally`. También es válida la combinación `try` y `finally` sin ningún `except`.

⚠️ ¡A evitar! ⚠️

En general, se recomienda evitar las capturas demasiado amplias y, especialmente, `except:` sin indicar un tipo. Estas formas pueden capturar problemas que el programa no sabe resolver y ocultar errores lógicos. Además, `except:` también intercepta señales especiales, como la interrupción solicitada con Ctrl+C.

Una captura con `except Exception` puede ser apropiada en el límite general de una aplicación si existe una respuesta concreta, como registrar el problema y volver a lanzarlo. Sin embargo, no conviene usarla de manera predeterminada para ignorar cualquier fallo.

Siempre que sea posible, se debe capturar únicamente la excepción específica que se espera y se sabe manejar:

```python
try:
    numero = int(texto)
except ValueError:
    print("El texto no representa un número entero")
```

7 Excepciones a medida

Los tipos de excepciones *built-in* son suficientes para muchísimos problemas. Sin embargo, si una situación posee un significado propio del dominio de nuestra aplicación, podemos definir una excepción personalizada.

7.1 Las excepciones son objetos

Los tipos de excepción forman una jerarquía de clases. Una versión muy simplificada es:

```
BaseException
├── KeyboardInterrupt
├── SystemExit
└── Exception
    ├── ArithmeticError
    │   └── ZeroDivisionError
    ├── LookupError
    │   ├── IndexError
    │   └── KeyError
    ├── OSError
    │   └── FileNotFoundError
    ├── TypeError
    └── ValueError
```

`BaseException` es la raíz de la jerarquía, pero las excepciones que representan fallos ordinarios de un programa heredan de `Exception`. Otras, como `KeyboardInterrupt` y `SystemExit`, representan señales con las que se solicita interrumpir o terminar la ejecución. Esta es la razón por la que una captura general con `except Exception` no las intercepta, mientras que un `except:` sin tipo sí puede hacerlo.

Una excepción personalizada es también una clase. Habitualmente hereda de `Exception` o de alguna de sus subclases, y no directamente de `BaseException`:

```
class SaldoInsuficienteError(Exception):
    pass
```

Por convención, los nombres de las excepciones terminan en `Error`. Al heredar de `Exception`, la nueva clase participa del mismo mecanismo de lanzamiento y captura que las excepciones incorporadas.

```
class CuentaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def extraer(self, monto):
        if monto > self.saldo:
            raise SaldoInsuficienteError("No hay saldo suficiente")
```

```
        self.saldo -= monto
        return monto
```

Podemos capturarla por su tipo:

```
cuenta = CuentaBancaria("Ana", 5000)

try:
    cuenta.extraer(7000)
except SaldoInsuficienteError as error:
    print("Extracción rechazada:", error)
```

```
Extracción rechazada: No hay saldo suficiente
```

La clase personalizada permite distinguir la falta de saldo de cualquier otro `ValueError` que pudiera ocurrir.

7.2 Jerarquía de excepciones

La herencia permite agrupar problemas relacionados:

```
class CuentaError(Exception):
    pass

class SaldoInsuficienteError(CuentaError):
    pass

class MontoInvalidoError(CuentaError):
    pass
```

Una parte del programa puede capturar un problema concreto:

```
try:
    cuenta.extraer(monto)
except SaldoInsuficienteError:
    print("No alcanza el saldo disponible")
```

Otra parte puede manejar de la misma manera cualquier problema relacionado con una cuenta:

```
try:
    realizar_operacion()
except CuentaError as error:
    print("La operación fue rechazada:", error)
```

Como `SaldoInsuficienteError` y `MontoInvalidoError` heredan de `CuentaError`, ambas coinciden con ese último `except`.

7.3 Orden de excepciones por especificidad

Python evalúa los bloques `except` de arriba hacia abajo. Por eso las clases hijas deben aparecer antes que sus clases padre:

```
try:
    realizar_operacion()
except SaldoInsuficienteError:
    print("Puede intentar con un monto menor")
except CuentaError:
    print("Ocurrió otro problema relacionado con la cuenta")
```

Si `CuentaError` apareciera primero, también capturaría los objetos de tipo `SaldoInsuficienteError`, y el bloque específico nunca llegaría a ejecutarse.

8 *Warnings*

Las **advertencias** o *warnings* informan situaciones sospechosas, obsoletas o potencialmente problemáticas que no impiden necesariamente continuar. A diferencia de una excepción no capturada, un *warning* se muestra y el programa normalmente sigue ejecutándose.

8.1 Crear una advertencia

Las advertencias se emiten mediante el módulo `warnings`:

```
import warnings

def convertir_pesos(monto, cotizacion=None):
    if cotizacion is None:
        warnings.warn(
            "No se indicó una cotización; se usará el valor 1",
            UserWarning,
        )
        cotizacion = 1

    return monto / cotizacion
```

```
resultado = convertir_pesos(1000)
print("Resultado:", resultado)
```

La advertencia se muestra, pero el cálculo continúa.

> **Nota**
>
> Usar `print("Advertencia: ...")` solamente escribe texto. `warnings.warn(...)` crea una advertencia que incluye una categoría y que puede ser filtrada, convertida en excepción o registrada por las herramientas que ejecutan el programa.

8.2 Categorías frecuentes

Algunosco *built-in* son:

Categoría	Uso habitual
`UserWarning`	Advertencias generales creadas por una aplicación.
`DeprecationWarning`	Una característica quedará obsoleta o será eliminada.
`FutureWarning`	Un comportamiento cambiará en el futuro.
`RuntimeWarning`	Una situación dudosa detectada durante la ejecución.
`SyntaxWarning`	Una construcción sintácticamente válida parece sospechosa.

Algunas categorías, como `DeprecationWarning`, pueden estar ocultas por defecto según dónde se originen. Por eso no debe asumirse que toda advertencia siempre será visible.

8.3 Advertencias personalizadas

Las categorías de *warning* también son clases y pueden extenderse mediante herencia:

```
class CotizacionWarning(UserWarning):
    pass
```

```
warnings.warn(
    "La cotización tiene más de 24 horas",
    CotizacionWarning,
)
```

Una categoría propia permite aplicar filtros únicamente a las advertencias de nuestra aplicación.

8.4 Filtrar advertencias

`warnings.simplefilter` permite controlar cómo se tratarán:

```
import warnings

warnings.simplefilter("always", CotizacionWarning)
```

Algunas acciones disponibles son:

- `"always"`: muestra la advertencia cada vez que se emite;

- `"default"`: muestra la primera aparición de cada advertencia en cada línea del código;
- `"module"`: muestra la primera aparición de cada advertencia dentro de cada módulo, sin importar desde qué línea se emita;
- `"once"`: muestra la primera aparición de cada advertencia durante el programa, sin importar el módulo o la línea donde se emita;
- `"ignore"`: no muestra las advertencias coincidentes;
- `"error"`: las convierte en excepciones.

Para decidir si dos advertencias son la misma, Python compara su mensaje y su categoría. La diferencia entre `"default"`, `"module"` y `"once"` es si también tiene en cuenta la línea, el módulo o ninguna ubicación, respectivamente.

Por ejemplo, durante las pruebas puede ser útil convertir una categoría en excepción:

```
warnings.simplefilter("error", CotizacionWarning)

try:
    warnings.warn(
        "La cotización tiene más de 24 horas",
        CotizacionWarning,
    )
except CotizacionWarning as error:
    print("La prueba detectó una advertencia:", error)
```

> ℹ *¿Warning* o excepción? 🤔
>
> Conviene lanzar una **excepción** cuando la operación no puede cumplir lo solicitado. Una **advertencia** es más apropiada cuando la operación todavía puede producir un resultado razonable, pero existe una condición que la persona programadora debería conocer.
>
> Silenciar todas las advertencias indiscriminadamente puede ocultar cambios incompatibles o resultados potencialmente incorrectos.

9 Tópicos avanzados 🧩

9.1 Agregar información a la excepción

Como cualquier objeto, una excepción puede guardar estado en sus atributos:

```
class SaldoInsuficienteError(Exception):
    def __init__(self, saldo, monto):
        self.saldo = saldo
        self.monto = monto
        mensaje = (
            f"Saldo insuficiente: disponible ${saldo}, "
```

```
            f"solicitado ${monto}"
        )
        super().__init__(mensaje)
```

El llamado a `super().__init__(mensaje)` inicializa la parte heredada de `Exception` y hace que el mensaje se muestre correctamente al convertir la excepción en texto.

```
class CuentaBancaria:
    def __init__(self, titular, saldo=0):
        self.titular = titular
        self.saldo = saldo

    def extraer(self, monto):
        if monto > self.saldo:
            raise SaldoInsuficienteError(self.saldo, monto)

        self.saldo -= monto
        return monto
```

Quien captura la excepción puede usar tanto el mensaje como sus atributos:

```
try:
    cuenta.extraer(7000)
except SaldoInsuficienteError as error:
    faltante = error.monto - error.saldo
    print(error)
    print("Faltan $", faltante, sep="")
```

9.2 Propagación avanzada

En ocasiones una función necesita realizar una acción al capturar una excepción, pero no puede o no debe resolverla por completo.

9.2.a Relanzar una excepción

La instrucción `raise` sin argumentos, utilizada dentro de un `except`, vuelve a lanzar la misma excepción:

```
def procesar_archivo(ruta):
    try:
        with open(ruta, encoding="utf-8") as archivo:
            return archivo.read()
    except FileNotFoundError:
        print("Se intentó abrir:", ruta)
        raise
```

La función registra información útil y después permite que `FileNotFoundError` continúe propagándose. El código de un nivel superior conserva el tipo, el mensaje y el *traceback* originales.

9.2.b Transformar y encadenar excepciones

Una capa de la aplicación puede querer expresar un problema técnico mediante una excepción más significativa para su dominio. `raise ... from ...` conserva la relación entre ambas:

```
class ConfiguracionError(Exception):
    pass

def leer_puerto(texto):
    try:
        return int(texto)
    except ValueError as error_original:
        raise ConfiguracionError(
            "El puerto configurado debe ser un número entero"
        ) from error_original
```

Si se llama a `leer_puerto("ochenta")`, el *traceback* muestra el `ValueError` original y luego el `ConfiguracionError` que se produjo como consecuencia directa. Esto agrega contexto sin borrar la causa inicial.

También existe `raise NuevaExcepcion(...) from None`, que oculta el contexto anterior en el *traceback* mostrado. Debe usarse con cuidado, porque simplifica el mensaje visible pero también puede eliminar información valiosa para diagnosticar el problema.

9.3 Administrar *warnings* temporalmente

Los filtros configurados con `warnings.simplefilter()` modifican el comportamiento general del programa. Si sólo queremos aplicar un filtro durante una operación determinada, podemos usar `warnings.catch_warnings()` como administrador de contexto:

```
with warnings.catch_warnings():
    warnings.simplefilter("ignore", CotizacionWarning)
    warnings.warn("Cotización desactualizada", CotizacionWarning)
```

Dentro del bloque `with`, las advertencias de tipo `CotizacionWarning` son ignoradas. Al finalizar el bloque, `catch_warnings()` restaura automáticamente la configuración anterior, incluso si durante la ejecución se produce una excepción. De esta manera, el cambio no afecta al resto del programa.

También es posible guardar temporalmente las advertencias emitidas en una lista. Esta variante es especialmente útil para comprobar su contenido durante una prueba:

```
with warnings.catch_warnings(record=True) as advertencias:
    warnings.simplefilter("always")
    warnings.warn("Cotización desactualizada", CotizacionWarning)
```

```
    advertencia = advertencias[0]
    print(advertencia.message)
    print(advertencia.category.__name__)
```

```
Cotización desactualizada
CotizacionWarning
```

El argumento `record=True` evita que las advertencias se muestren de la manera habitual y agrega un registro por cada una a la lista `advertencias`. Cada registro permite consultar, entre otros datos, el mensaje, la categoría, el archivo y la línea donde se originó.